

Logical Inference

Forward & Backward Chaining

Sec 1: Logic &
Entailment

Sec 1 : Reasoning
Types

Sec 1 : Expert
Systems

Sec 1 : FOL

Sec 1 : Forward
Chaining

Sec 1 : Backward
Chaining

Contents Overview

Logical Inference & Entailment

Sec 1

Propositions, Connectives, Modus Ponens

Types of Reasoning

Sec 2

Deductive, Inductive, Abductive

Expert Systems

Sec 3

Components, Production Rules, Real-World Systems

First-Order Logic (FOL)

Sec 4

Quantifiers, Horn Clauses, Translation

Forward Chaining

Sec 5

Data-Driven Reasoning — 5 Full Examples

Backward Chaining

Sec 6

Goal-Driven Reasoning — 5 Full Examples

Section: 1

Logical Inference & Entailment

Propositions • Connectives • Modus Ponens • Entailment

What is Logical Inference?

In everyday life we constantly draw conclusions from available information.



Dark clouds

→ It will rain



Battery at 2%

→ Phone will die soon



AI Systems

→ Must reason the same way

DEFINITION — Logic

Logic is the formal science of correct reasoning. It provides a precise language for expressing knowledge and rules for deriving new true statements from existing ones. In AI, the machine can reason step by step — just like a human expert.

Propositions: The Building Blocks

DEFINITION — Proposition

A proposition is any declarative sentence that is either TRUE or FALSE — never both, never neither. A question, command, or opinion is NOT a proposition.

✓ Valid Propositions

"It is raining today"

→ TRUE or FALSE. This is a proposition.

"Ahmed has a fever"

→ TRUE or FALSE. This is a proposition.

" $2 + 2 = 4$ "

→ TRUE. This is a proposition.

✗ NOT Propositions

"What time is it?"

A question — no truth value.

"Please close the door"

A command — no truth value.

"This statement is false"

A paradox — violates TRUE/FALSE rule.

Logical Connectives

Symbol	Name	Plain English	True When...
$\neg P$	NOT P	NOT P	P is false
$P \wedge Q$	AND	P AND Q	Both P and Q are true
$P \vee Q$	OR	P OR Q	At least one of P, Q is true
$P \rightarrow Q$	IF...THEN	IF P THEN Q	FALSE only when P=true but Q=false
$P \leftrightarrow Q$	IFF	P if and only if Q	P and Q have the same truth value

⚠ The Implication ($P \rightarrow Q$) is the MOST IMPORTANT connective — every rule in an expert system is an implication!

The Implication $P \rightarrow Q$ (Most Important!)

The implication $P \rightarrow Q$ is read: "IF P THEN Q". Every rule in an expert system is an implication. Understanding when it is true or false is critical.

P (Condition)	Q (Conclusion)	$P \rightarrow Q$ Result	What it means
True	True	TRUE ✓	Rule held — condition met, conclusion followed
True	False	FALSE ✗	Rule VIOLATED — condition met but conclusion did NOT follow
False	True	TRUE ✓	Rule not triggered — does not apply
False	False	TRUE ✓	Rule not triggered — same as above

Key: Only FALSE when condition fires (P=true) but conclusion fails (Q=false). All other cases are TRUE.

Entailment & Key Inference Rules

DEFINITION — Entailment ($KB \models \alpha$)

KB entails α if and only if in EVERY possible world where all facts in KB are true, α is also true. α cannot possibly be false if KB is true.

Modus Ponens

$$P \rightarrow Q, P \vdash Q$$

If "IF it rains THEN ground is wet" is true, and "It is raining" is true $\rightarrow$ "The ground is wet". Core of forward chaining.

Modus Tollens

$$P \rightarrow Q, \neg Q \vdash \neg P$$

"IF it rains THEN ground is wet".
"Ground is NOT wet". Therefore: "It is NOT raining". The Contrapositive.

Hypothetical Syllogism

$$P \rightarrow Q, Q \rightarrow R \vdash P \rightarrow R$$

"IF smoke THEN fire" and "IF fire THEN alarm". Therefore: "IF smoke THEN alarm". Chains rules together.

Activity 1.1 — Identify the Propositions

Identify which of the following are valid propositions and which are not. Explain why for each one.

- (a) "All computers in Lab 3 are connected to the internet."
- (b) "Is the printer working?"
- (c) "Please restart the server."
- (d) "If temperature exceeds 100°C, the system shuts down."
- (e) "This statement is false."

Solution — Activity 1.1

- (a) VALID PROPOSITION — It is a declarative sentence: either True or False.
- (b) NOT a proposition — It is a question; questions have no truth value.
- (c) NOT a proposition — It is a command; commands cannot be True or False.
- (d) VALID PROPOSITION — It is a declarative conditional with a clear truth value.
- (e) NOT a standard proposition — It is the "Liar Paradox". If it is true it is false, and vice versa — it violates the rule that a proposition must be unambiguously True or False.

Activity 1.2 — Calculate Truth Values

Given: $P = \text{TRUE}$ and $Q = \text{FALSE}$

Calculate the truth value of each expression and explain your reasoning:

- (a) $\neg P$
- (b) $P \wedge Q$
- (c) $P \vee Q$
- (d) $P \rightarrow Q$
- (e) $Q \rightarrow P$

Solution — Activity 1.2

(a) $\neg P = \neg \text{TRUE} = \text{FALSE}$

(b) $P \wedge Q = \text{TRUE} \wedge \text{FALSE} = \text{FALSE}$ (both must be true; Q is false)

(c) $P \vee Q = \text{TRUE} \vee \text{FALSE} = \text{TRUE}$ (at least one is true; P is true)

(d) $P \rightarrow Q = \text{TRUE} \rightarrow \text{FALSE} = \text{FALSE}$ (condition TRUE but conclusion FALSE — rule VIOLATED)

(e) $Q \rightarrow P = \text{FALSE} \rightarrow \text{TRUE} = \text{TRUE}$ (condition is FALSE — rule is never triggered, so it holds)

⚠ Key Insight: (d) is FALSE but (e) is TRUE. The DIRECTION of an implication matters enormously!

Activity 1.3 — Apply Modus Ponens

Apply the inference rules step by step:

Rule 1: IF a vehicle has no fuel THEN it will not start.

Rule 2: IF a vehicle will not start THEN the driver will be late.

Fact: Sara's car has no fuel.

Using Hypothetical Syllogism followed by Modus Ponens, what can you conclude about Sara?

Solution — Activity 1.3

Step 1 — Apply Hypothetical Syllogism:

Rule 1: $\text{NoFuel} \rightarrow \text{WillNotStart}$

Rule 2: $\text{WillNotStart} \rightarrow \text{DriverLate}$

Combined: $\text{NoFuel} \rightarrow \text{DriverLate}$ (new derived rule)

Step 2 — Apply Modus Ponens:

We have: $\text{NoFuel} \rightarrow \text{DriverLate}$ (from Step 1)

We know: $\text{NoFuel}(\text{Sara})$ is TRUE (given fact)

Therefore: $\text{DriverLate}(\text{Sara})$ is TRUE

Conclusion: Sara will be late.

Section 2

Types of Reasoning

Deductive • Inductive • Abductive

Three Types of Reasoning



Deductive

*General rules + specific facts →
Guaranteed specific conclusion*

Example:

All humans are mortal. Socrates is human.
∴ Socrates is mortal.

AI Use: Expert systems, logic engines
Certainty: 100% guaranteed



Inductive

*Many specific observations → Probable
general rule*

Example:

Observed 1000 white swans → concluded
all swans are white (then found a black
one!)

AI Use: Machine learning, neural nets
Certainty: Probable only



Abductive

Observation → Most likely explanation

Example:

Grass is wet → It probably rained (could be
a sprinkler — best guess only)

AI Use: Medical diagnosis, NLP
Certainty: Best guess

Comparison Table

Feature	Deductive	Inductive	Abductive
Starting point	General rules	Specific examples	An observation
Goal	Specific conclusion	General rule	Best explanation
Certainty	100% guaranteed	Probable only	Best guess
AI Use	Expert systems, logic	ML, neural nets	Diagnosis, NLP

Activity 2.1 — Classify the Reasoning

Classify each scenario as Deductive, Inductive, or Abductive. Give a one-sentence justification.

- (a) A student observes that every time he studies the night before, he fails the test.
He concludes: "Studying the night before causes failure."
- (b) The rule says: "All students who miss 3 or more classes are barred from the exam."
Zara has missed 4 classes. Therefore Zara is barred.
- (c) The server response time is 8 seconds (normally 0.2 seconds).
You conclude: "The database must be overloaded."

Solution — Activity 2.1

- (a) **INDUCTIVE** — He is observing repeated specific instances and forming a general rule. The conclusion is probable, not certain.
- (b) **DEDUCTIVE** — He is applying a known general rule to a specific verified fact. The conclusion is 100% guaranteed.
- (c) **ABDUCTIVE** — He is given an observation (slow response) and proposing the **BEST EXPLANATION** (overloaded database). Other explanations exist but this is most plausible.

Activity 2.2 — Traffic System Analysis

A city traffic system has the rule:

"IF road_blocked AND alternative_route_available THEN divert_traffic."

The system observes:

- Road_A is blocked
- No alternative route is available

- (a) Which reasoning type does this system use?
- (b) Does the rule fire? Construct the truth table for this implication.
- (c) What is the LIMITATION of abductive reasoning compared to this system?



Solution — Activity 2.2

- (a) DEDUCTIVE reasoning — the system applies a known IF-THEN rule to specific facts.
- (b) Road_Blocked(A) = TRUE, AlternativeRoute = FALSE
 $\text{TRUE} \wedge \text{FALSE} = \text{FALSE}$ — the conjunction is FALSE, so the rule does NOT fire.
The system does NOT divert traffic because no alternative is available.
- (c) Abductive reasoning gives the BEST GUESS, not a guaranteed conclusion.
Deductive reasoning gives CERTAIN outcomes given verified facts.

Section 3

Expert Systems

Rule-Based Systems • Production Rules • Real-World Applications

What is an Expert System?

DEFINITION — Expert System

An expert system is an AI program that captures the knowledge and reasoning ability of a human expert in a specific domain. It uses a Knowledge Base (what the expert knows) and an Inference Engine (how the expert thinks) to answer questions — even without the human expert present.

1 Knowledge Base

Stores FACTS (known truths) and RULES (IF-THEN expert knowledge). Also maintains Working Memory — a dynamic store of everything currently known.

2 Inference Engine

The "thinking" part. Scans the KB for rules whose conditions are satisfied, fires them to derive new facts, and repeats until a goal is reached.

3 Explanation Module

Explains WHY the system reached a conclusion by tracing which rules fired. Critical in medicine, law, and finance — accountability matters.

Production Rules & Famous Expert Systems

Production Rule Format

IF <condition1> AND <condition2> AND ... THEN <conclusion>

This is the standard format for encoding expert knowledge as machine-executable rules.

System	Year	Domain	Achievement
MYCIN	1976	Medicine	Diagnosed blood infections using 600 rules. Performed as well as specialist physicians.
XCON	1980	Engineering	Configured DEC VAX computers using 10,000+ rules. Saved \$40 million per year.
DENDRAL	1969	Chemistry	Identified molecular structures from spectrometer data. First major AI success story.

Activity 3.1 — Academic Probation Expert System

A university expert system decides if a student should be placed on academic probation:

Rules:

R1: IF GPA < 2.0 AND FailedSubjects > 2 THEN OnProbation

R2: IF OnProbation AND MissedOrientation THEN Expelled

Student Record: GPA = 1.7 | Failed subjects = 3 | Attended orientation = NO

Write the production rules in FOL, list the facts, then trace what conclusions the inference engine reaches.

Solution — Activity 3.1

RULES:

R1: IF $\text{LowGPA}(x) \wedge \text{FailedMultiple}(x)$ $\rightarrow \text{OnProbation}(x)$

R2: IF $\text{OnProbation}(x) \wedge \text{MissedOrientation}(x)$ $\rightarrow \text{Expelled}(x)$

FACTS: $\text{LowGPA}(\text{student})$ | $\text{FailedMultiple}(\text{student})$ | $\text{MissedOrientation}(\text{student})$

TRACE:

Pass 1: R1 checks — $\text{LowGPA} \checkmark$ AND $\text{FailedMultiple} \checkmark \rightarrow \text{R1 FIRES} \rightarrow \text{OnProbation}(\text{student})$ added

Pass 2: R2 checks — $\text{OnProbation} \checkmark$ AND $\text{MissedOrientation} \checkmark \rightarrow \text{R2 FIRES} \rightarrow \text{Expelled}(\text{student})$ added

Pass 3: No more rules can fire. STOP.

CONCLUSION: The student is EXPELLED.

Activity 3.2 — Expert System vs Neural Network

Explain why an expert system is MORE SUITABLE than a neural network for a hospital medication approval system.

Use the 3 components of an expert system in your answer.

(Think about when you NEED to explain WHY a decision was made — accountability and auditability are key.)



Solution — Activity 3.2

1. Knowledge Base: Rules written explicitly by medical experts (e.g., "IF drug A AND drug B THEN fatal_interaction"). Doctors can review, verify, and update the exact rules being used.
2. Inference Engine: Each decision is traced through specific rules. If the system says "Do not prescribe X", it is because Rule 7 fired, requiring condition Y and Z.
3. Explanation Module: Tells the pharmacist EXACTLY why it flagged an interaction. A neural network outputs a probability but CANNOT explain.

Why neural networks are unsuitable: (a) black boxes — no explanation, (b) may lack training data for rare interactions, (c) a wrong decision cannot be easily corrected.

Conclusion: In life-critical domains, expert systems are the correct choice.

Section 4

First-Order Logic (FOL)

Quantifiers • Predicates • Horn Clauses • Translation Practice

Why FOL? Core Components

With simple propositional logic, you need separate statements per person: "Ahmed-has-fever", "Sara-has-fever". With FOL, write Fever(x) once — x stands for any person.

Component	Example	What it means
Constants	Ahmed, London, 42	Specific named objects in the world
Variables	x, y, z	Placeholders — can stand for any object
Predicates	Fever(x), Loves(x,y)	Properties of objects, or relationships between them
$\forall x$ (For All)	$\forall x \text{ Human}(x) \rightarrow \text{Mortal}(x)$	The statement is true for EVERY object x
$\exists x$ (There Exists)	$\exists x \text{ Student}(x) \wedge \text{Smart}(x)$	There is AT LEAST ONE object x satisfying this

The Critical Quantifier Rule (MEMORISE)

⚠️ MEMORISE THIS

Use $\forall$ (for all) with $\rightarrow$ (implication)

Use $\exists$ (there exists) with $\wedge$ (AND)

This is the most common mistake students make in exams.

✅ CORRECT

$\forall x \text{ Human}(x) \rightarrow \text{Mortal}(x)$

Every human is mortal

✅ CORRECT

$\exists x \text{ Student}(x) \wedge \text{Smart}(x)$

Some student is smart

❌ WRONG

$\forall x \text{ Human}(x) \wedge \text{Mortal}(x)$

Every object is BOTH human AND mortal — incorrect!

English → FOL Translation Practice

English Sentence	FOL Formula	Key Rule
Every student studies	$\forall x \text{ Student}(x) \rightarrow \text{Studies}(x)$	$\forall$ with $\rightarrow$
Some students are hardworking	$\exists x \text{ Student}(x) \wedge \text{Hardworking}(x)$	$\exists$ with $\wedge$
No dog can fly	$\forall x \text{ Dog}(x) \rightarrow \neg \text{CanFly}(x)$	$\forall$ + NOT in conclusion
Every patient with fever & cough has flu	$\forall x [\text{Fever}(x) \wedge \text{Cough}(x)] \rightarrow \text{Flu}(x)$	$\forall$ + compound condition
Only doctors can prescribe	$\forall x \text{ Prescribes}(x) \rightarrow \text{Doctor}(x)$	"Only A does B" = If B then A

Horn Clauses: FOL in Rule Engines

DEFINITION — Horn Clause

A Horn Clause has at most ONE positive conclusion (the head) and any number of conditions (the body).

Format: $\text{Head} \leftarrow \text{Body1} \wedge \text{Body2} \wedge \dots \wedge \text{BodyN}$

Both Prolog and CLIPS use Horn Clauses because they make reasoning efficient and decidable.

Fact (unconditionally true)

`Employed(Ahmed) ←`

(No body – always true regardless of situation)

Rule (head $\leftarrow$ body)

`EligibleForLoan(x) ←
 Employed(x)
 $\wedge$ AgeOver21(x)
 $\wedge$ GoodCredit(x)`



Activity 4.1 — English to FOL Translation

Translate each sentence into a correct FOL formula. State which quantifier you used and why.

- (a) Every registered voter must be over 18.
- (b) Some employees earn a bonus.
- (c) No machine can think.
- (d) Every employee who works overtime and has good reviews gets a raise.
- (e) At least one student passed all 5 subjects.

Solution — Activity 4.1

- (a) $\forall x \text{ RegisteredVoter}(x) \rightarrow \text{Over18}(x)$ Used $\forall$ with $\rightarrow$: IF registered voter THEN over 18.
- (b) $\exists x \text{ Employee}(x) \wedge \text{EarnsBonus}(x)$ Used $\exists$ with $\wedge$: "some" means at least one.
- (c) $\forall x \text{ Machine}(x) \rightarrow \neg \text{CanThink}(x)$ Used $\forall$ with $\rightarrow$ and negation. "No machine" = "every machine does NOT".
- (d) $\forall x [\text{Employee}(x) \wedge \text{WorksOvertime}(x) \wedge \text{GoodReviews}(x)] \rightarrow \text{GetsRaise}(x)$
Used $\forall$ with compound condition — multiple conditions joined with $\wedge$.
- (e) $\exists x \text{ Student}(x) \wedge \text{PassedAll5}(x)$ Used $\exists$ with $\wedge$. "At least one" is existential.

Activity 4.2 — Smart Home Horn Clauses

Write the following in Horn Clause format:

"If motion is detected and it is night time and the alarm is armed, then trigger the alarm."

"The alarm is armed." (a fixed fact)

"It is night time." (a fixed fact)

Then show how the inference engine derives new facts when MotionDetected(Room3) is received.

Solution — Activity 4.2

HORN CLAUSES:

Fact 1: AlarmArmed $\leftarrow$ (no body — unconditionally true)
Fact 2: NightTime $\leftarrow$ (no body — unconditionally true)
Rule: TriggerAlarm(r) $\leftarrow$ MotionDetected(r) $\wedge$ NightTime $\wedge$ AlarmArmed

INFERENCE TRACE:

WM starts: { AlarmArmed, NightTime }
Sensor sends: MotionDetected(Room3)
WM becomes: { AlarmArmed, NightTime, MotionDetected(Room3) }
Rule check: MotionDetected(Room3) ✓ NightTime ✓ AlarmArmed ✓
All 3 met $\rightarrow$ Rule FIRES $\rightarrow$ TriggerAlarm(Room3) added

CONCLUSION: The alarm is triggered for Room 3.

Section 5

Forward Chaining

Data-Driven Reasoning • 5 Full Examples

Forward Chaining: Definition & Algorithm

DEFINITION — Forward Chaining

Starts with KNOWN FACTS and keeps applying rules to derive NEW facts, until either a goal is reached or no more rules can fire. It asks: "Given what I already know, what NEW things can I figure out?" Works forward from DATA to CONCLUSIONS.

MATCH

1

Look at every rule. Check whether ALL conditions of that rule are currently in Working Memory.

SELECT

2

From rules whose conditions are all satisfied (the conflict set), pick one to fire.

FIRE

3

Apply the selected rule. Write its conclusion as a NEW FACT in Working Memory.

TERMINATE?

4

Goal reached → STOP. No new facts added → FIXPOINT → STOP. Otherwise go to Step 1.

Critical Skill: Identifying Rules vs Facts

CRITICAL SKILL

Before you can do forward or backward chaining, you must know HOW to extract rules and facts from a real-world description.



RULE

A conditional relationship between two or more things.
Applies to CATEGORIES of objects — uses VARIABLES (x).

Test: "Does this apply to EVERY X in a category?" → RULE

Example: "IF an animal has fur AND gives milk THEN it is a mammal"



FACT

A specific piece of known information about a specific individual. Uses CONSTANTS (specific names). Facts are GIVEN — not derived.

Test: "Is this about a specific NAMED individual?" → FACT

Example: "The animal has fur" (about THIS animal specifically)

Example 5.1: Animal Classification



PROBLEM STATEMENT

We observe an unknown animal. The animal has fur on its body, produces milk to feed its young, and has black and orange stripes on its body.

Based on biological classification knowledge, determine what type of animal this is.

Ex 5.1 (Animal) — Step 1: Extract Facts & Rules

Item	Type	Reason for Classification
The animal has fur	FACT	Specific observation about THIS animal. Describes a concrete property of one individual.
The animal produces milk	FACT	Specific observation about this specific individual animal. Directly given.
The animal has stripes	FACT	Directly observed property of this specific animal. Given, not derived.
IF HasFur AND GivesMilk THEN Mammal	RULE	General biological principle applying to ALL animals — uses variable x.
IF Mammal AND HasStripes THEN Tiger	RULE	General biological rule applicable to any mammal with stripes. Category-level conditional.

Ex 5.1 (Animal) — Step 2: FOL Representation

FOL Rules

R1: $\forall x \text{ HasFur}(x) \wedge \text{GivesMilk}(x)$
 $\rightarrow \text{Mammal}(x)$

"For every animal x: if x has
fur AND gives milk $\rightarrow$ mammal"

R2: $\forall x \text{ Mammal}(x) \wedge \text{HasStripes}(x)$
 $\rightarrow \text{Tiger}(x)$

"For every animal x: if x is a
mammal AND has stripes $\rightarrow$ tiger"

Facts & Initial WM

F1: $\text{HasFur}(\text{animal})$

F2: $\text{GivesMilk}(\text{animal})$

F3: $\text{HasStripes}(\text{animal})$

Initial WM:

{ HasFur, GivesMilk, HasStripes }

Ex 5.1 (Animal) — Forward Chaining Trace

Pass 1 Check R1	R1: HasFur(animal) ✓ GivesMilk(animal) ✓ BOTH satisfied. R1 FIRES. → NEW FACT: Mammal(animal) added. WM = { HasFur, GivesMilk, HasStripes, Mammal }
Pass 2 Check R2	R2: Mammal(animal) ✓ (just derived) HasStripes(animal) ✓ BOTH satisfied. R2 FIRES. → NEW FACT: Tiger(animal) added. WM = { HasFur, GivesMilk, HasStripes, Mammal, Tiger }
Pass 3 Fixpoint	Rescan all rules: R1 → Mammal already in WM. R2 → Tiger already in WM. No rule can add any new fact. FIXPOINT REACHED. STOP.

Ex 5.1 (Animal) — Conclusion



CONCLUSION

The animal is a TIGER.

Chain: HasFur + GivesMilk $\rightarrow$ Mammal (Pass 1, R1 fires)

Mammal + HasStripes $\rightarrow$ Tiger (Pass 2, R2 fires)

Key Insight: R2 could NOT fire until R1 first derived "Mammal". Derived facts unlock further rules — that is the essence of forward chaining.

Ex 5.1 (Animal) — Forward Chaining Diagram (Part 1 of 2)



Ex 5.1 (Animal) — Forward Chaining Diagram (Part 2 of 2)



Conditions NOT all met --> RULE SKIPPED

Note: HasFeathers and CanFly are NOT in WM. R2 cannot fire. Skip.

PASS 3 Rule: R3

Mammal(x)
In WM

HasStripes(x)
In WM



FIRES --> NEW FACT ADDED: Tiger(x)

Note: Mammal was derived in Pass 1. HasStripes is an original fact in WM. R3 fires.

PASS 4 Rule: R4

Mammal(x)
In WM

HasSpots(x)
NOT in WM



Conditions NOT all met --> RULE SKIPPED

Note: HasSpots is not in WM. R4 cannot fire. All rules scanned. FIXPOINT reached.

Example 5.2: Medical Diagnosis



PROBLEM STATEMENT

A patient named Ahmed visits a clinic. The doctor observes: high fever, coughing, and sharp chest pain.

Medical knowledge:

- Any patient with fever and cough has an infection.
- If a patient has an infection and also chest pain, the infection is serious.
- Any patient with a serious infection must be hospitalized.
- If a patient has fever and a rash, that indicates an allergy.

Ex 5.2 (Medical) — Step 1: Extract Facts & Rules

Item	Type	Reason for Classification
Ahmed has a high fever	FACT	Directly observed about this specific patient. Uses the constant 'Ahmed'.
Ahmed is coughing	FACT	Specific clinical observation about Ahmed. Given, not derived.
Ahmed has chest pain	FACT	Specific symptom observed in this specific patient. Given.
IF fever AND cough THEN infection	RULE	General medical principle applying to ANY patient. Uses variable p.
IF infection AND chest pain THEN serious	RULE	General medical rule applicable to any patient with these conditions.
IF serious infection THEN hospitalize	RULE	General medical protocol for any patient with a serious infection.
IF fever AND rash THEN allergy	RULE	General rule — Ahmed has NO rash, so this will NOT fire.

Ex 5.2 (Medical) — Step 2: FOL Representation

FOL Rules

R1: $\forall p \text{ Fever}(p) \wedge \text{Cough}(p) \rightarrow \text{Infection}(p)$
R2: $\forall p \text{ Infection}(p) \wedge \text{ChestPain}(p) \rightarrow \text{SeriousInfection}(p)$
R3: $\forall p \text{ SeriousInfection}(p) \rightarrow \text{Hospitalize}(p)$
R4: $\forall p \text{ Fever}(p) \wedge \text{Rash}(p) \rightarrow \text{Allergy}(p)$

Facts & Initial WM

F1: $\text{Fever}(\text{Ahmed})$
F2: $\text{Cough}(\text{Ahmed})$
F3: $\text{ChestPain}(\text{Ahmed})$
(Rash – NOT given, R4 won't fire)

Initial WM:
{ Fever, Cough, ChestPain }

Ex 5.2 (Medical) — Forward Chaining Trace

Pass 1 Check R1	R1: Fever(Ahmed) ✓ Cough(Ahmed) ✓ R1 FIRES. → NEW FACT: Infection(Ahmed) added. WM = { Fever, Cough, ChestPain, Infection }
Pass 1 R2,R3,R4	R2: Infection not yet in WM at start of pass. CANNOT FIRE. R3: SeriousInfection not in WM. Cannot fire. R4: Rash(Ahmed) not in WM. Cannot fire.
Pass 2 Check R2	R2: Infection(Ahmed) ✓ (derived in Pass 1) ChestPain(Ahmed) ✓ R2 FIRES. → NEW FACT: SeriousInfection(Ahmed) added.
Pass 3 Check R3	R3: SeriousInfection(Ahmed) ✓ (derived in Pass 2) R3 FIRES. → NEW FACT: Hospitalize(Ahmed) added.

Ex 5.2 (Medical) — Forward Chaining Trace

Pass 4
Fixpoint

All rules rescanned. No rule can add any new fact. R4 still cannot fire.
FIXPOINT REACHED. STOP.

Ex 5.2 (Medical) — Conclusion



CONCLUSION

Ahmed must be HOSPITALIZED.

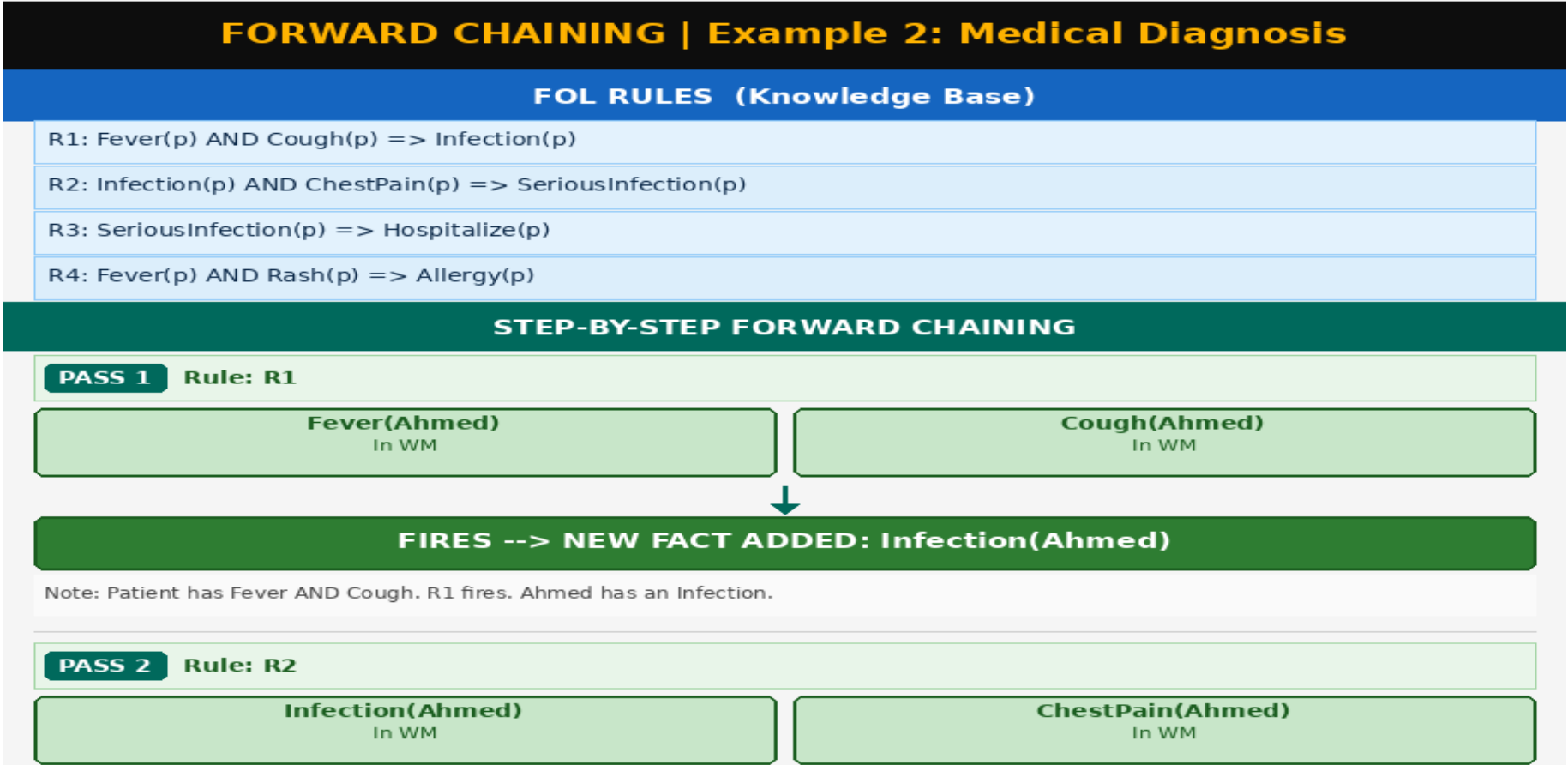
Chain: Fever + Cough → Infection (Pass 1)

Infection + ChestPain → SeriousInfection (Pass 2)

SeriousInfection → Hospitalize (Pass 3)

Note: R4 (Allergy rule) was in the Knowledge Base but NEVER fired — no Rash fact existed.

Ex 5.2 (Medical) — Forward Chaining Diagram (Part 1 of 2)



Ex 5.2 (Medical) — Forward Chaining Diagram (Part 2 of 2)



FIRES --> NEW FACT ADDED: SeriousInfection(Ahmed)

Note: Infection derived in Pass 1. ChestPain also in WM. R2 fires.

PASS 3 Rule: R3

SeriousInfection(Ahmed)
In WM



FIRES --> NEW FACT ADDED: Hospitalize(Ahmed)

Note: SeriousInfection derived in Pass 2. R3 fires. Ahmed must be Hospitalized.

PASS 4 Rule: R4

Fever(Ahmed)
In WM

Rash(Ahmed)
NOT in WM



Conditions NOT all met --> RULE SKIPPED

Note: Rash is not in WM. R4 (Allergy) cannot fire. FIXPOINT reached. STOP.

Example 5.3: Loan Eligibility



PROBLEM STATEMENT

A bank processes a loan application for Sara. Sara is employed, 27 years old (over 21), has good credit, and earns PKR 1,800,000 annually (above the PKR 600,000 threshold).

Bank policy: employed + over21 → basically eligible. basically eligible + good credit → fully eligible. fully eligible + high income → premium loan. basically eligible + NO good credit → micro loan only.

Ex 5.3 (Loan) — Step 1: Extract Facts & Rules

Item	Type	Reason for Classification
Sara is employed	FACT	Specific verified information about the individual Sara. Named constant.
Sara is over 21	FACT	Age verified for this specific person. Given evidence.
Sara has good credit	FACT	Specific verified credit record attribute for Sara.
Sara earns over threshold	FACT	Specific financial fact about Sara. Verified income figure.
IF Employed AND AgeOver21 THEN BasicEligible	RULE	Bank policy applying to ANY applicant. Uses variable x.
IF BasicEligible AND GoodCredit THEN FullEligible	RULE	General bank policy applicable to any applicant.
IF FullEligible AND HighIncome THEN PremiumLoan	RULE	General eligibility tier rule for any customer.
IF BasicEligible AND NOT GoodCredit THEN MicroLoan	RULE	Alternative pathway — will NOT fire because Sara HAS good credit.

Ex 5.3 (Loan) — Step 2: FOL Representation

FOL Rules

R1: $\forall x \text{ Employed}(x) \wedge \text{AgeOver21}(x)$
 $\rightarrow \text{BasicEligible}(x)$
R2: $\forall x \text{ BasicEligible}(x) \wedge \text{GoodCredit}(x)$
 $\rightarrow \text{FullEligible}(x)$
R3: $\forall x \text{ FullEligible}(x) \wedge \text{IncomeHigh}(x)$
 $\rightarrow \text{PremiumLoan}(x)$
R4: $\forall x \text{ BasicEligible}(x) \wedge \neg \text{GoodCredit}(x)$
 $\rightarrow \text{MicroLoanOnly}(x)$

Facts & Initial WM

F1: $\text{Employed}(\text{Sara})$
F2: $\text{AgeOver21}(\text{Sara})$
F3: $\text{GoodCredit}(\text{Sara})$
F4: $\text{IncomeHigh}(\text{Sara})$
(GoodCredit present $\rightarrow$ R4 blocked)

Initial WM:
{ Employed, AgeOver21,
 GoodCredit, IncomeHigh }

Ex 5.3 (Loan) — Forward Chaining Trace

Pass 1 Check R1	R1: Employed(Sara) ✓ AgeOver21(Sara) ✓ R1 FIRES. → NEW FACT: BasicEligible(Sara) added.
Pass 2 Check R2	R2: BasicEligible(Sara) ✓ (derived) GoodCredit(Sara) ✓ R2 FIRES. → NEW FACT: FullEligible(Sara) added.
Pass 2 Check R4	R4: BasicEligible ✓ BUT ¬GoodCredit(Sara) is FALSE (Sara HAS good credit). Negation condition fails. R4 CANNOT FIRE.
Pass 3 Check R3	R3: FullEligible(Sara) ✓ (derived) IncomeHigh(Sara) ✓ R3 FIRES. → NEW FACT: PremiumLoan(Sara) added.

Ex 5.3 (Loan) — Forward Chaining Trace

Pass 4
Fixpoint

No new facts can be derived. FIXPOINT REACHED. STOP.

Ex 5.3 (Loan) — Conclusion



CONCLUSION

Sara qualifies for a PREMIUM LOAN.

Chain: Employed + AgeOver21 $\rightarrow$ BasicEligible (R1)

BasicEligible + GoodCredit $\rightarrow$ FullEligible (R2)

FullEligible + IncomeHigh $\rightarrow$ PremiumLoan (R3)

R4 was blocked because Sara has good credit — the negation $\neg$ GoodCredit was false.

Ex 5.3 (Loan) — Forward Chaining Diagram (Part 1 of 2)

FORWARD CHAINING | Example 3: Loan Eligibility

FOL RULES (Knowledge Base)

R1: $\text{Employed}(x) \text{ AND } \text{AgeOver21}(x) \Rightarrow \text{BasicEligible}(x)$

R2: $\text{BasicEligible}(x) \text{ AND } \text{GoodCredit}(x) \Rightarrow \text{FullEligible}(x)$

R3: $\text{FullEligible}(x) \text{ AND } \text{IncomeHigh}(x) \Rightarrow \text{PremiumLoan}(x)$

R4: $\text{BasicEligible}(x) \text{ AND } \text{NOT GoodCredit}(x) \Rightarrow \text{MicroLoanOnly}(x)$

STEP-BY-STEP FORWARD CHAINING

PASS 1 Rule: R1

Employed(Sara)
In WM

AgeOver21(Sara)
In WM



FIRES --> NEW FACT ADDED: BasicEligible(Sara)

Note: Sara is employed and over 21. R1 fires. Sara is BasicEligible.

PASS 2 Rule: R2

BasicEligible(Sara)
In WM

GoodCredit(Sara)
In WM



Ex 5.3 (Loan) — Forward Chaining Diagram (Part 2 of 2)

FIRES --> NEW FACT ADDED: FullEligible(Sara)

Note: BasicEligible derived in Pass 1. GoodCredit is a given fact. R2 fires.

PASS 2 Rule: R4 (negation check)

BasicEligible(Sara)
In WM

NOT GoodCredit
GoodCredit IS in WM



Negation NOT satisfied --> RULE SKIPPED

Note: R4 requires NOT GoodCredit. But Sara HAS good credit. Negation fails. R4 cannot fire.

PASS 3 Rule: R3

FullEligible(Sara)
In WM

IncomeHigh(Sara)
In WM



FIRES --> NEW FACT ADDED: PremiumLoan(Sara)

Note: FullEligible derived in Pass 2. IncomeHigh is a given fact. R3 fires.

PASS 4 Fixpoint Check

All rules scanned. No new facts can be added to WM. FIXPOINT reached. STOP.

Example 5.4: Student Scholarship



PROBLEM STATEMENT

A university evaluates student Ali. Ali scored 91 in his final exam and has good attendance.

University policy: $\text{score} \geq 85 \rightarrow \text{high achiever}$. $\text{high achiever} + \text{good attendance} \rightarrow \text{distinction}$. $\text{distinction} + \text{needs-based} \rightarrow \text{scholarship}$. $\text{distinction} + \text{NOT needs-based} \rightarrow \text{merit award}$.

Ali has NOT applied for needs-based support.

Ex 5.4 (Scholarship) — Step 1: Extract Facts & Rules

Item	Type	Reason for Classification
Ali scored 91	FACT	Specific exam result for Ali. A concrete measured value. Given data.
Ali has good attendance	FACT	Specific attendance record for Ali. A named individual's attribute.
Ali is NOT needs-based	FACT	Absence of needs-based application for this specific student.
IF score $\geq$ 85 THEN HighAchiever	RULE	General university policy for ALL students. Uses variable x and a threshold.
IF HighAchiever AND GoodAttendance THEN Distinction	RULE	Another general policy applying to any student. Uses a variable.
IF Distinction AND NeedsBased THEN Scholarship	RULE	General award rule — NeedsBased must be a fact for this student.
IF Distinction AND NOT NeedsBased THEN MeritAward	RULE	Alternative award rule. The negation $\neg$ NeedsBased will be satisfied for Ali.

Ex 5.4 (Scholarship) — Step 2: FOL Representation

FOL Rules

- R1: $\forall x \text{ Score}(x,s) \wedge s \geq 85$
 $\rightarrow \text{HighAchiever}(x)$
- R2: $\forall x \text{ HighAchiever}(x) \wedge \text{GoodAttend}(x)$
 $\rightarrow \text{Distinction}(x)$
- R3: $\forall x \text{ Distinction}(x) \wedge \text{NeedsBased}(x)$
 $\rightarrow \text{Scholarship}(x)$
- R4: $\forall x \text{ Distinction}(x) \wedge \neg \text{NeedsBased}(x)$
 $\rightarrow \text{MeritAward}(x)$

Facts & Initial WM

- F1: $\text{Score}(\text{Ali}, 91)$
F2: $\text{GoodAttend}(\text{Ali})$
 $\text{NeedsBased}(\text{Ali})$ NOT in WM
- Initial WM:
{ $\text{Score}(\text{Ali}, 91)$, $\text{GoodAttend}(\text{Ali})$ }

Ex 5.4 (Scholarship) — Forward Chaining Trace

Pass 1 Check R1	R1: $\text{Score}(\text{Ali}, 91) \checkmark$ AND $91 \geq 85 \checkmark$ R1 FIRES. → NEW FACT: $\text{HighAchiever}(\text{Ali})$ added.
Pass 2 Check R2	R2: $\text{HighAchiever}(\text{Ali}) \checkmark$ (derived) $\text{GoodAttend}(\text{Ali}) \checkmark$ R2 FIRES. → NEW FACT: $\text{Distinction}(\text{Ali})$ added.
Pass 3 R3 vs R4	R3: $\text{Distinction}(\text{Ali}) \checkmark$ $\text{NeedsBased}(\text{Ali}) \times$ — NOT in WM. R3 CANNOT FIRE. R4: $\text{Distinction}(\text{Ali}) \checkmark$ $\neg \text{NeedsBased}(\text{Ali}) \checkmark$ (absent → negation TRUE) R4 FIRES. → NEW FACT: $\text{MeritAward}(\text{Ali})$ added.
Pass 4 Fixpoint	No more rules can fire. FIXPOINT REACHED. STOP.

Ex 5.4 (Scholarship) — Conclusion

CONCLUSION

Ali receives a MERIT AWARD (not a scholarship).

Chain: $\text{Score}(91) \geq 85 \rightarrow \text{HighAchiever}$ (R1)

$\text{HighAchiever} + \text{GoodAttend} \rightarrow \text{Distinction}$ (R2)

$\text{Distinction} + \neg \text{NeedsBased} \rightarrow \text{MeritAward}$ (R4)

R3 was blocked because NeedsBased was never a fact for Ali. R4 fired instead.

Ex 5.4 (Scholarship) — Forward Chaining Diagram (Part 1 of 2)

FORWARD CHAINING | Example 4: Student Scholarship Decision

FOL RULES (Knowledge Base)

R1: $\text{Score}(x,s) \text{ AND } s \geq 85 \Rightarrow \text{HighAchiever}(x)$

R2: $\text{HighAchiever}(x) \text{ AND } \text{GoodAttend}(x) \Rightarrow \text{Distinction}(x)$

R3: $\text{Distinction}(x) \text{ AND } \text{NeedsBased}(x) \Rightarrow \text{Scholarship}(x)$

R4: $\text{Distinction}(x) \text{ AND } \text{NOT NeedsBased}(x) \Rightarrow \text{MeritAward}(x)$

STEP-BY-STEP FORWARD CHAINING

PASS 1 Rule: R1

Score(Ali,91)
In WM

91 $\geq$ 85
Threshold Met



FIRES --> NEW FACT ADDED: HighAchiever(Ali)

Note: Score is 91, which satisfies the ≥ 85 threshold. R1 fires.

PASS 2 Rule: R2

HighAchiever(Ali)
In WM

GoodAttend(Ali)
In WM

Ex 5.4 (Scholarship) — Forward Chaining Diagram (Part 2 of 2)

FIRES --> NEW FACT ADDED: Distinction(Ali)

Note: HighAchiever derived in Pass 1. GoodAttendance is a given fact. R2 fires.

PASS 3 Rule: R3

Distinction(Ali)
In WM

NeedsBased(Ali)
NOT in WM

Conditions NOT all met --> RULE SKIPPED

Note: Distinction is derived but NeedsBased is NOT in WM. R3 (Scholarship) cannot fire.

PASS 3 Rule: R4

Distinction(Ali)
In WM

NOT NeedsBased
NeedsBased absent

FIRES --> NEW FACT ADDED: MeritAward(Ali)

Note: Distinction derived. NeedsBased is absent from WM so NOT NeedsBased is TRUE. R4 fires.

CONCLUSION: Ali receives a MERIT AWARD

Example 5.5: Network Intrusion Detection



PROBLEM STATEMENT

A network security system monitors traffic from source IP-X.

Sensors report: traffic volume is unusually high | IP-X is not in the registered database (unknown) | IP-X is scanning multiple ports | data payload contains a known malicious signature

Security policy: high traffic + unknown IP → suspicious. suspicious + port scan → attacker. attacker + malicious payload → active threat. active threat → block and alert.

Ex 5.5 (Network) — Step 1: Extract Facts & Rules

Item	Type	Reason for Classification
IP-X has high traffic	FACT	Specific real-time sensor reading about this IP. Observed data point.
IP-X is an unknown IP	FACT	A lookup result about this specific IP. Concrete fact.
IP-X is doing port scanning	FACT	Detected behavior of this specific IP. Observed and verified.
IP-X has a malicious payload	FACT	Sensor result about this specific IP's data. Direct observation.
IF HighTraffic AND UnknownIP THEN SuspiciousSource	RULE	General security policy for any source. Uses variable s.
IF SuspiciousSource AND PortScan THEN Attacker	RULE	General threat escalation rule for any suspicious source.
IF Attacker AND MaliciousPayload THEN ActiveThreat	RULE	General rule for any attacker with payload evidence.
IF ActiveThreat THEN BlockAndAlert	RULE	General response protocol for any confirmed active threat.

Ex 5.5 (Network) — Step 2: FOL Representation

FOL Rules

- R1: $\forall s \text{ HighTraffic}(s) \wedge \text{UnknownIP}(s) \rightarrow \text{SuspiciousSource}(s)$
R2: $\forall s \text{ SuspiciousSource}(s) \wedge \text{PortScan}(s) \rightarrow \text{Attacker}(s)$
R3: $\forall s \text{ Attacker}(s) \wedge \text{MaliciousPayload}(s) \rightarrow \text{ActiveThreat}(s)$
R4: $\forall s \text{ ActiveThreat}(s) \rightarrow \text{BlockAndAlert}(s)$

Facts & Initial WM

- F1: $\text{HighTraffic}(\text{IP-X})$
F2: $\text{UnknownIP}(\text{IP-X})$
F3: $\text{PortScan}(\text{IP-X})$
F4: $\text{MaliciousPayload}(\text{IP-X})$

Initial WM:
{ HighTraffic, UnknownIP,
PortScan, MaliciousPayload }

Ex 5.5 (Network) — Forward Chaining Trace

Pass 1 Check R1	R1: HighTraffic(IP-X) ✓ UnknownIP(IP-X) ✓ R1 FIRES. → SuspiciousSource(IP-X) added.
Pass 2 Check R2	R2: SuspiciousSource(IP-X) ✓ (derived) PortScan(IP-X) ✓ R2 FIRES. → Attacker(IP-X) added.
Pass 3 Check R3	R3: Attacker(IP-X) ✓ (derived) MaliciousPayload(IP-X) ✓ R3 FIRES. → ActiveThreat(IP-X) added.
Pass 4 Check R4	R4: ActiveThreat(IP-X) ✓ (derived) R4 FIRES. → BlockAndAlert(IP-X) added.

Ex 5.5 (Network) — Forward Chaining Trace

Pass 5
Fixpoint

All four rules have fired. No rule uses BlockAndAlert as a condition.
FIXPOINT REACHED. STOP.

Ex 5.5 (Network) — Conclusion



CONCLUSION

IP-X is BLOCKED and an alert is sent.

Complete 4-rule chain:

HighTraffic + UnknownIP → SuspiciousSource (R1)

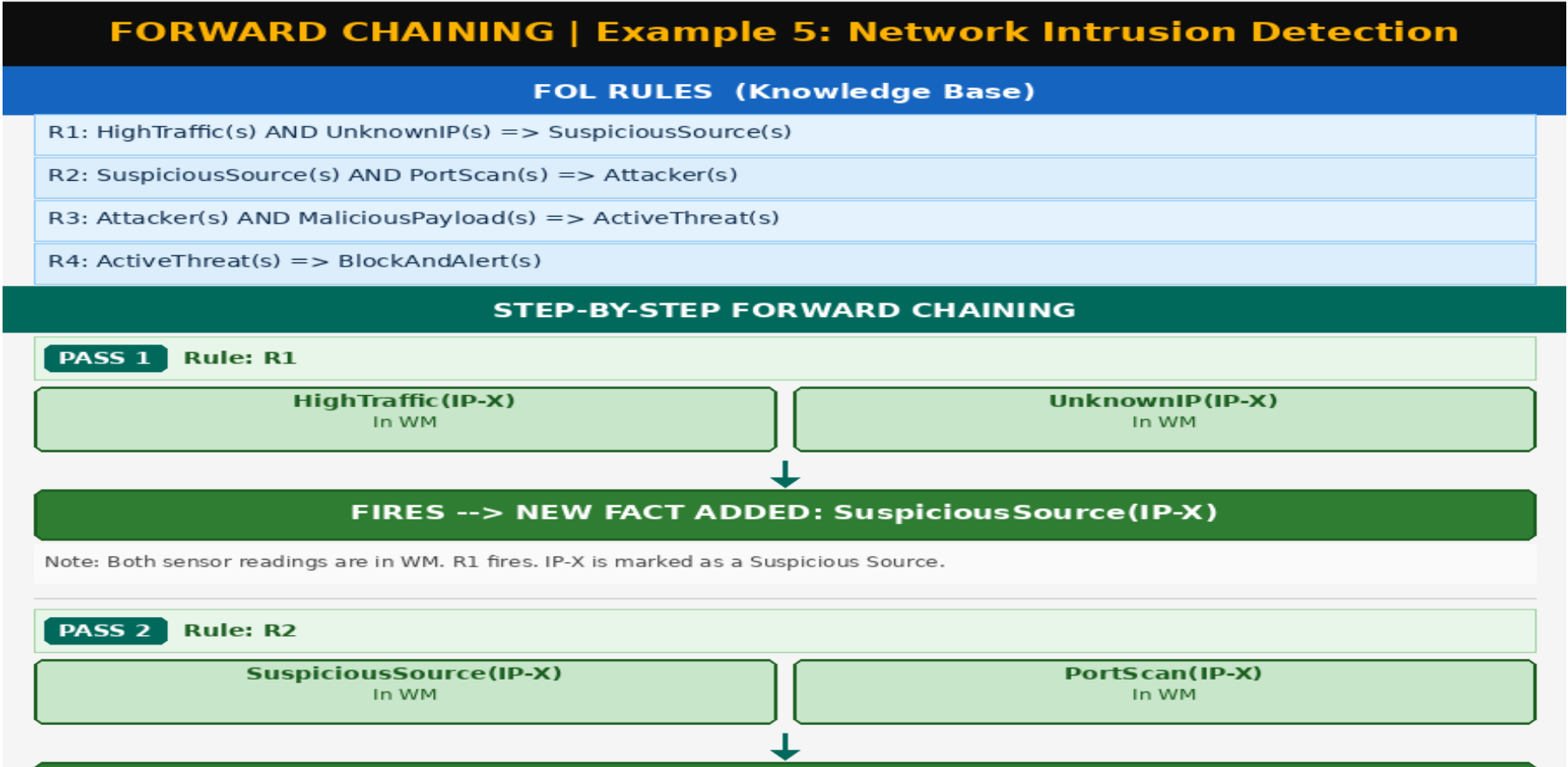
SuspiciousSource + PortScan → Attacker (R2)

Attacker + MaliciousPayload → ActiveThreat (R3)

ActiveThreat → BlockAndAlert (R4)

Every rule in the Knowledge Base contributed. This is a COMPLETE forward chain.

Ex 5.5 (Network) — Forward Chaining Diagram (Part 1 of 2)



Ex 5.5 (Network) — Forward Chaining Diagram (Part 2 of 2)

FIRES --> NEW FACT ADDED: Attacker(IP-X)

Note: SuspiciousSource derived in Pass 1. PortScan is a given sensor fact. R2 fires.

PASS 3 Rule: R3

Attacker(IP-X)
In WM

MaliciousPayload(IP-X)
In WM



FIRES --> NEW FACT ADDED: ActiveThreat(IP-X)

Note: Attacker derived in Pass 2. MaliciousPayload is a given sensor fact. R3 fires.

PASS 4 Rule: R4

ActiveThreat(IP-X)
In WM



FIRES --> NEW FACT ADDED: BlockAndAlert(IP-X)

Note: ActiveThreat derived in Pass 3. R4 fires. All 4 rules fired in a perfect chain.

PASS 5 Fixpoint Check

No rule can fire anymore. BlockAndAlert is in WM but no rule uses it as a condition. FIXPOINT. STOP.

CONCLUSION: IP-X is BLOCKED and an alert is raised

Section 6

Backward Chaining

Goal-Driven Reasoning • 5 Full Examples

Backward Chaining: Definition & Algorithm

DEFINITION — Backward Chaining

Starts with a GOAL (the conclusion we want to prove) and works BACKWARDS to find whether the facts support it. It asks: "In order to prove this goal, what conditions do I need? Can I prove those conditions?" It is goal-driven — only explores rules relevant to the goal.

1

SET GOAL

Take the top-level question/goal and place it on the goal stack.

2

CHECK WM

Is the current goal already a known fact? If YES → PROVED. Move to next sub-goal.

3

FIND RULE

Search the KB for any rule whose HEAD matches the current goal.

4

PUSH SUB-GOALS

Take all CONDITIONS of that matching rule and push them as new sub-goals.

5

RECURSE

Go back to Step 2 for each sub-goal. Sub-goals may generate their own sub-goals.

6

CONCLUDE

All sub-goals proved → goal PROVED. No rule found AND not a fact → goal FAILS.

Forward vs Backward Chaining: Key Insight

Forward Chaining

Start: ALL known facts in WM
Strategy: Fire all applicable rules
Direction: Data → Conclusions
Asks: "What can I discover?"
Like: A doctor running all possible tests
Use: When you want ALL possible conclusions
Stops at: Fixpoint (no new facts)

Backward Chaining

Start: ONE specific goal
Strategy: Find rules with head = goal
Direction: Goal → Required conditions
Asks: "Can I prove THIS specific thing?"
Like: A lawyer proving one specific case
Use: When you have ONE question to answer
Stops at: Goal proved or goal fails

SAME Knowledge Base. SAME Facts. SAME Rules. The only difference is the DIRECTION of reasoning.

Example 6.1: Animal Classification



PROBLEM STATEMENT

We observe an animal that has fur, gives milk, and has stripes.

GOAL QUESTION: "Is this animal a Tiger?"

Use backward chaining to prove or disprove this goal.

Ex 6.1 (Animal (Backward)) — FOL Representation

FOL Rules

R1: $\forall x \text{ HasFur}(x) \wedge \text{GivesMilk}(x)$
 $\rightarrow \text{Mammal}(x)$
R2: $\forall x \text{ Mammal}(x) \wedge \text{HasStripes}(x)$
 $\rightarrow \text{Tiger}(x)$

Facts & Goal

F1: $\text{HasFur}(\text{animal})$
F2: $\text{GivesMilk}(\text{animal})$
F3: $\text{HasStripes}(\text{animal})$

WM: { HasFur, GivesMilk,
 HasStripes }

GOAL: PROVE $\text{Tiger}(\text{animal})$

Ex 6.1 (Animal) — Backward Chaining Trace

Step 1 Goal: Tiger(x)	Is Tiger(x) in WM? NO. Search KB for HEAD = Tiger(x). Found R2. PUSH SUB-GOALS: [Mammal(x), HasStripes(x)]
Step 2 Sub-Goal: Mammal(x)	Is Mammal(x) in WM? NO. Search KB for HEAD = Mammal(x). Found R1. PUSH SUB-GOALS: [HasFur(x), GivesMilk(x)]
Step 3 Sub-Goal: HasFur(x)	Is HasFur(x) in WM? YES ✓ PROVED directly from facts.
Step 4 Sub-Goal: GivesMilk(x)	Is GivesMilk(x) in WM? YES ✓ PROVED directly from facts.

Ex 6.1 (Animal) — Backward Chaining Trace

Step 5 Return: Mammal(x)	Both conditions of R1 proved: HasFur ✓ GivesMilk ✓ R1 fires. MAMMAL(x) is PROVED. ✓
Step 6 Sub-Goal: HasStripes(x)	Is HasStripes(x) in WM? YES ✓ PROVED directly from facts.
Step 7 Return: Tiger(x)	Both conditions of R2 proved: Mammal ✓ HasStripes ✓ R2 fires. TIGER(x) is PROVED. ✓

Ex 6.1 (Animal) — Conclusion



GOAL PROVED

YES — the animal is confirmed to be a TIGER.

Backward chain:

Goal: $\text{Tiger}(x) \leftarrow R2 \leftarrow [\text{Mammal}(x), \text{HasStripes}(x)]$

Sub-Goal $\text{Mammal}(x) \leftarrow R1 \leftarrow [\text{HasFur } \checkmark, \text{GivesMilk } \checkmark]$

$\text{HasStripes } \checkmark$ (found directly in WM)

Ex 6.1 (Animal) — Backward Chaining Diagram (Part 1 of 2)

BACKWARD CHAINING | Example 1: Is the Animal a Tiger?

FOL RULES (Knowledge Base)

R1: $\text{HasFur}(x) \text{ AND } \text{GivesMilk}(x) \Rightarrow \text{Mammal}(x)$

R2: $\text{Mammal}(x) \text{ AND } \text{HasStripes}(x) \Rightarrow \text{Tiger}(x)$

STARTING GOAL (What we want to prove)

GOAL: $\text{Tiger}(x)$

BACKWARD CHAINING - Step by Step

FIND RULE Goal: $\text{Tiger}(x)$

--> Found R2: needs $\text{Mammal}(x) \text{ AND } \text{HasStripes}(x)$

Sub-goals pushed: $\text{Mammal}(x) \text{ AND } \text{HasStripes}(x)$



Note: Tiger is not a known fact. We search for a rule whose head is Tiger. R2 matches.

FIND RULE Goal: $\text{Mammal}(x)$

--> Found R1: needs $\text{HasFur}(x) \text{ AND } \text{GivesMilk}(x)$

Sub-goals pushed: $\text{HasFur}(x) \text{ AND } \text{GivesMilk}(x)$



Note: Mammal is not a known fact. R1 can prove it. Push its conditions as sub-goals.

Ex 6.1 (Animal) — Backward Chaining Diagram (Part 2 of 2)

CHECK FACTS Goal: HasFur(x)

--> HasFur(x) IS in Working Memory (given as initial fact)



Note: HasFur is a directly given fact. Sub-goal proved immediately.

CHECK FACTS Goal: GivesMilk(x)

--> GivesMilk(x) IS in Working Memory (given as initial fact)



Note: GivesMilk is a directly given fact. Sub-goal proved.

PROVED ✓ Goal: Mammal(x)

--> All conditions of R1 proved => Mammal(x) is now PROVED



Note: Both HasFur and GivesMilk proved. R1 fires. Mammal is derived.

CHECK FACTS Goal: HasStripes(x)

--> HasStripes(x) IS in Working Memory (given as initial fact)



Note: HasStripes is a directly given fact. Sub-goal proved.

PROVED ✓ Goal: Tiger(x)

--> All conditions of R2 proved => Tiger(x) is PROVED



Note: Both Mammal and HasStripes proved. R2 fires. Goal achieved!

Example 6.2: Loan Eligibility



PROBLEM STATEMENT

A bank AI system receives a query: "Is Ahmed's loan approved?"

Known facts: Ahmed is employed | Ahmed is 25 years old (over 21) | Ahmed has a good credit score

Bank policy: any employed person over 21 is basically eligible. Any basically eligible person with good credit gets their loan approved.

Ex 6.2 (Loan (Backward)) — FOL Representation

FOL Rules

R1: $\forall x \text{ Employed}(x) \wedge \text{AgeOver21}(x)$
 $\rightarrow \text{BasicEligible}(x)$
R2: $\forall x \text{ BasicEligible}(x) \wedge \text{GoodCredit}(x)$
 $\rightarrow \text{LoanApproved}(x)$

Facts & Goal

F1: $\text{Employed}(\text{Ahmed})$
F2: $\text{AgeOver21}(\text{Ahmed})$
F3: $\text{GoodCredit}(\text{Ahmed})$

WM: { Employed , AgeOver21 ,
 GoodCredit }

GOAL: PROVE $\text{LoanApproved}(\text{Ahmed})$

Ex 6.2 (Loan) — Backward Chaining Trace

Step 1 Goal: LoanApproved	Is LoanApproved in WM? NO. Found R2. Conditions: BasicEligible(Ahmed) AND GoodCredit(Ahmed). PUSH SUB-GOALS: [BasicEligible(Ahmed), GoodCredit(Ahmed)]
Step 2 Sub-Goal: BasicEligible	Is BasicEligible in WM? NO. Found R1. Conditions: Employed(Ahmed) AND AgeOver21(Ahmed). PUSH SUB-GOALS: [Employed(Ahmed), AgeOver21(Ahmed)]
Step 3 Employed(Ahmed)	Is Employed(Ahmed) in WM? YES ✓ PROVED directly.
Step 4 AgeOver21(Ahmed)	Is AgeOver21(Ahmed) in WM? YES ✓ PROVED directly.

Ex 6.2 (Loan) — Backward Chaining Trace

Step 5
Return: BasicEligible

Both R1 conditions proved: Employed ✓ AgeOver21 ✓
R1 fires. BASICELIGIBLE(Ahmed) PROVED. ✓

Step 6
GoodCredit(Ahmed)

Is GoodCredit(Ahmed) in WM? YES ✓
PROVED directly.

Step 7
Return: LoanApproved

Both R2 conditions proved: BasicEligible ✓ GoodCredit ✓
R2 fires. LOANAPPROVED(Ahmed) PROVED. ✓

Ex 6.2 (Loan) — Conclusion



GOAL PROVED

YES — Ahmed's loan is APPROVED.

Backward chain:

Goal: $\text{LoanApproved}(\text{Ahmed}) \leftarrow R2 \leftarrow [\text{BasicEligible}, \text{GoodCredit}]$

Sub-Goal $\text{BasicEligible}(\text{Ahmed}) \leftarrow R1 \leftarrow [\text{Employed} \checkmark, \text{AgeOver21} \checkmark]$

GoodCredit directly found in WM $\checkmark$

Ex 6.2 (Loan) — Backward Chaining Diagram (Part 1 of 2)

BACKWARD CHAINING | Example 2: Is Ahmed's Loan Approved?

FOL RULES (Knowledge Base)

R1: $\text{Employed}(x) \text{ AND } \text{AgeOver21}(x) \Rightarrow \text{BasicEligible}(x)$

R2: $\text{BasicEligible}(x) \text{ AND } \text{GoodCredit}(x) \Rightarrow \text{LoanApproved}(x)$

STARTING GOAL (What we want to prove)

GOAL: $\text{LoanApproved}(\text{Ahmed})$

BACKWARD CHAINING - Step by Step

FIND RULE Goal: $\text{LoanApproved}(\text{Ahmed})$

--> Found R2: needs $\text{BasicEligible}(\text{Ahmed}) \text{ AND } \text{GoodCredit}(\text{Ahmed})$

Sub-goals pushed: $\text{BasicEligible}(\text{Ahmed}) \text{ AND } \text{GoodCredit}(\text{Ahmed})$



Note: LoanApproved is not a fact. R2 can prove it. Push its two conditions as sub-goals.

FIND RULE Goal: $\text{BasicEligible}(\text{Ahmed})$

--> Found R1: needs $\text{Employed}(\text{Ahmed}) \text{ AND } \text{AgeOver21}(\text{Ahmed})$

Sub-goals pushed: $\text{Employed}(\text{Ahmed}) \text{ AND } \text{AgeOver21}(\text{Ahmed})$



Note: BasicEligible is not a fact. R1 can prove it. Push its conditions.

Ex 6.2 (Loan) — Backward Chaining Diagram (Part 2 of 2)

CHECK FACTS Goal: Employed(Ahmed)

--> Employed(Ahmed) IS in Working Memory



Note: Ahmed is listed as employed in the known facts. Proved.

CHECK FACTS Goal: AgeOver21(Ahmed)

--> AgeOver21(Ahmed) IS in Working Memory



Note: Ahmed is over 21. This is a known fact. Proved.

PROVED ✓ Goal: BasicEligible(Ahmed)

--> R1 fully satisfied => BasicEligible(Ahmed) PROVED



Note: Both conditions of R1 proved. BasicEligible is now a derived fact.

CHECK FACTS Goal: GoodCredit(Ahmed)

--> GoodCredit(Ahmed) IS in Working Memory



Note: Good credit is a known fact. Proved immediately.

PROVED ✓ Goal: LoanApproved(Ahmed)

--> R2 fully satisfied => LoanApproved(Ahmed) PROVED



Note: Both BasicEligible and GoodCredit proved. R2 fires. Goal achieved!

Example 6.3: Medical Diagnosis



PROBLEM STATEMENT

A medical system is asked: "Does Sara need antibiotics?"

Sara's record: fever | coughing | fatigue | symptoms lasted more than 3 days

Medical knowledge: fever + cough + fatigue $\rightarrow$ flu. flu + symptoms >3 days $\rightarrow$ needs antibiotics.

Ex 6.3 (Medical (Backward)) — FOL Representation

FOL Rules

R1: $\forall p \text{ Fever}(p) \wedge \text{Cough}(p) \wedge \text{Fatigue}(p)$
 $\rightarrow \text{HasFlu}(p)$
R2: $\forall p \text{ HasFlu}(p) \wedge \text{DurationOver3Days}(p)$
 $\rightarrow \text{NeedsAntibiotics}(p)$

Facts & Goal

F1: $\text{Fever}(\text{Sara})$
F2: $\text{Cough}(\text{Sara})$
F3: $\text{Fatigue}(\text{Sara})$
F4: $\text{DurationOver3Days}(\text{Sara})$

WM: { Fever, Cough, Fatigue,
 DurationOver3Days }

GOAL: PROVE $\text{NeedsAntibiotics}(\text{Sara})$

Ex 6.3 (Medical) — Backward Chaining Trace

Step 1 Goal: NeedsAntibiotics	Is NeedsAntibiotics in WM? NO. Found R2. Conditions: HasFlu(Sara) AND DurationOver3Days(Sara). PUSH SUB-GOALS: [HasFlu(Sara), DurationOver3Days(Sara)]
Step 2 Sub-Goal: HasFlu	Is HasFlu in WM? NO. Found R1. Conditions: Fever, Cough, Fatigue. PUSH SUB-GOALS: [Fever(Sara), Cough(Sara), Fatigue(Sara)]
Step 3 Fever,Cough,Fatigue	Fever(Sara) in WM? YES ✓ Cough(Sara) in WM? YES ✓ Fatigue(Sara) in WM? YES ✓ All three PROVED directly from facts.
Step 4 Return: HasFlu	All 3 conditions of R1 proved: Fever ✓ Cough ✓ Fatigue ✓ R1 fires. HASFLU(Sara) PROVED. ✓

Ex 6.3 (Medical) — Backward Chaining Trace

Step 5
DurationOver3Days

Is DurationOver3Days(Sara) in WM? YES ✓
PROVED directly.

Step 6
Return:
NeedsAntibiotics

Both R2 conditions proved: HasFlu ✓ DurationOver3Days ✓
R2 fires. NEEDSANTIBIOTICS(Sara) PROVED. ✓

Ex 6.3 (Medical) — Conclusion



GOAL PROVED

YES — Sara NEEDS ANTIBIOTICS.

Backward chain:

Goal: NeedsAntibiotics $\leftarrow$ R2 $\leftarrow$ [HasFlu, DurationOver3Days]

Sub-Goal HasFlu $\leftarrow$ R1 $\leftarrow$ [Fever ✓, Cough ✓, Fatigue ✓]

DurationOver3Days ✓ (found directly in WM)

Note: R1 required THREE sub-goals to be proved — all found directly in WM.

Ex 6.3 (Medical) — Backward Chaining Diagram (Part 1 of 2)

BACKWARD CHAINING | Example 3: Does Sara Need Antibiotics?

FOL RULES (Knowledge Base)

R1: Fever(p) AND Cough(p) AND Fatigue(p) $\Rightarrow$ HasFlu(p)

R2: HasFlu(p) AND DurationOver3Days(p) $\Rightarrow$ NeedsAntibiotics(p)

STARTING GOAL (What we want to prove)

GOAL: NeedsAntibiotics(Sara)

BACKWARD CHAINING - Step by Step

FIND RULE Goal: NeedsAntibiotics(Sara)

--> Found R2: needs HasFlu(Sara) AND DurationOver3Days(Sara)

Sub-goals pushed: HasFlu(Sara) AND DurationOver3Days(Sara)



Note: NeedsAntibiotics is not a fact. R2 can prove it. Push its conditions.

FIND RULE Goal: HasFlu(Sara)

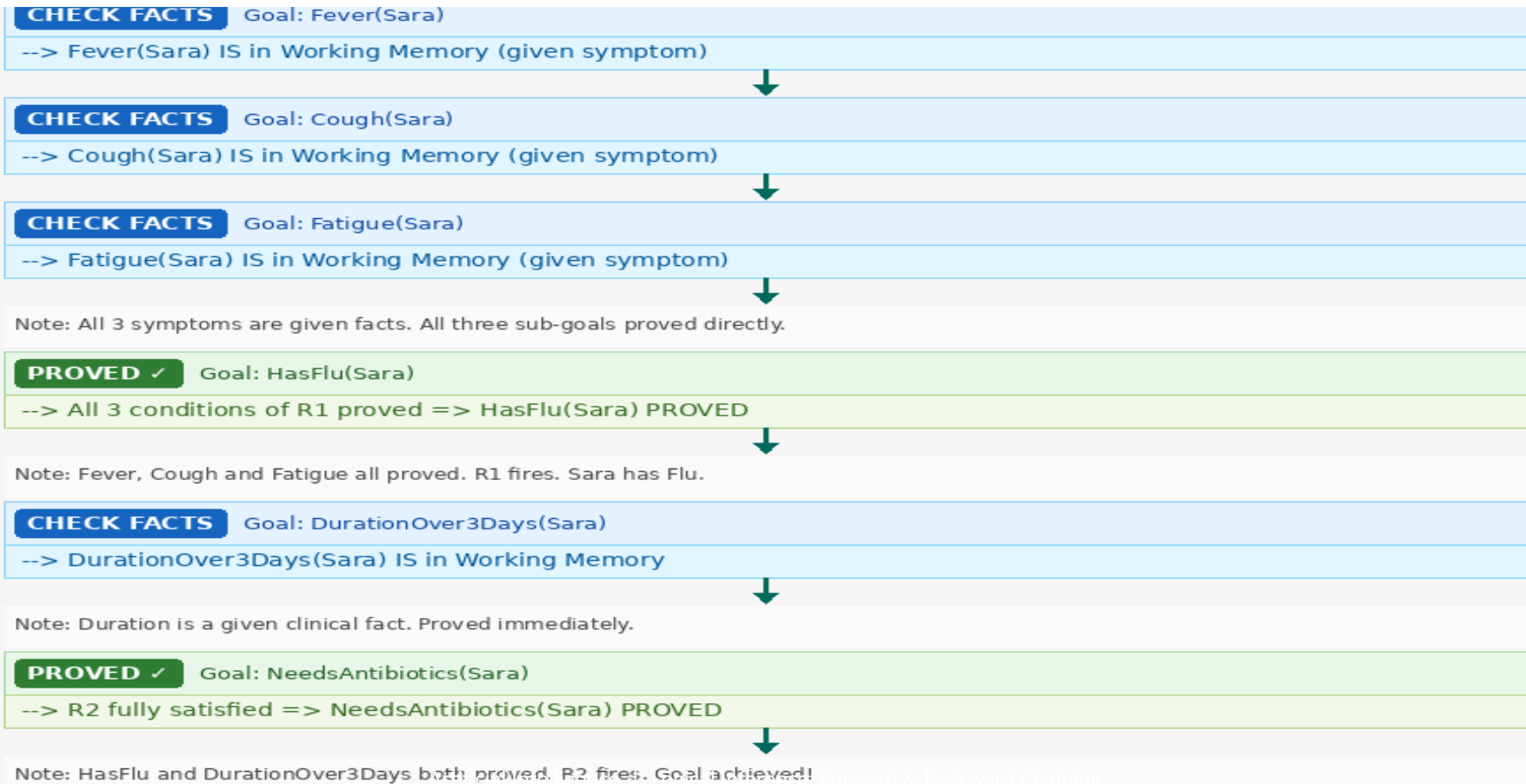
--> Found R1: needs Fever(Sara) AND Cough(Sara) AND Fatigue(Sara)

Sub-goals pushed: Fever(Sara) AND Cough(Sara) AND Fatigue(Sara)



Note: HasFlu is not a fact. R1 can prove it. Push all 3 conditions as sub-goals.

Ex 6.3 (Medical) — Backward Chaining Diagram (Part 2 of 2)



Example 6.4: Scholarship Award



PROBLEM STATEMENT

A university system is asked: "Is Omar a scholarship winner?"

Omar's record: scored 88 in exams | recognized as a student leader

University policy: $\text{score} \geq 80 \rightarrow \text{academic excellence}$. $\text{academic excellence} + \text{leader} \rightarrow \text{scholarship winner}$.

Ex 6.4 (Scholarship (Backward)) — FOL Representation

FOL Rules

R1: $\forall x \text{ Score}(x,s) \wedge s \geq 80$
 $\rightarrow \text{AcademicExcellence}(x)$
R2: $\forall x \text{ AcademicExcellence}(x) \wedge \text{Leader}(x)$
 $\rightarrow \text{ScholarshipWinner}(x)$

Facts & Goal

F1: $\text{Score}(\text{Omar}, 88)$
F2: $\text{Leader}(\text{Omar})$

WM: { $\text{Score}(\text{Omar}, 88)$, $\text{Leader}(\text{Omar})$ }

GOAL: PROVE
 $\text{ScholarshipWinner}(\text{Omar})$

Ex 6.4 (Scholarship) — Backward Chaining Trace

Step 1 Goal: ScholarshipWinner	Is ScholarshipWinner in WM? NO. Found R2. Conditions: AcademicExcellence(Omar) AND Leader(Omar). PUSH SUB-GOALS: [AcademicExcellence(Omar), Leader(Omar)]
Step 2 Sub-Goal: AcademicExcellence	Is AcademicExcellence in WM? NO. Found R1. Conditions: Score(Omar,s) AND $s \geq 80$. PUSH SUB-GOALS: [Score(Omar,88), $88 \geq 80$]
Step 3 Score(Omar,88)	Is Score(Omar,88) in WM? YES ✓ PROVED directly from facts.
Step 4 $88 \geq 80$	Arithmetic check: $88 \geq 80 \rightarrow \text{TRUE}$ ✓ PROVED by numerical verification.

Ex 6.4 (Scholarship) — Backward Chaining Trace

Step 5 Return: AcademicExcellence	Both R1 conditions proved: Score ✓ Threshold $88 \geq 80$ ✓ R1 fires. ACADEMICEXCELLENCE(Omar) PROVED. ✓
Step 6 Leader(Omar)	Is Leader(Omar) in WM? YES ✓ PROVED directly.
Step 7 Return: ScholarshipWinner	Both R2 conditions proved: AcademicExcellence ✓ Leader ✓ R2 fires. SCHOLARSHIPWINNER(Omar) PROVED. ✓

Ex 6.4 (Scholarship) — Conclusion



GOAL PROVED

YES — Omar WINS the scholarship.

Backward chain:

Goal: ScholarshipWinner $\leftarrow$ R2 $\leftarrow$ [AcademicExcellence, Leader]

Sub-Goal AcademicExcellence $\leftarrow$ R1 $\leftarrow$ [Score(88) ✓, $88 \geq 80$ ✓]

Leader ✓ (found directly in WM)

Note: The arithmetic check ($88 \geq 80$) is a SPECIAL TYPE of sub-goal — proved by computation, not by finding it in WM.

Ex 6.4 (Scholarship) — Backward Chaining Diagram (Part 1 of 2)

BACKWARD CHAINING | Example 4: Does Omar Win the Scholarship?

FOL RULES (Knowledge Base)

R1: $\text{Score}(x,s) \text{ AND } s \geq 80 \Rightarrow \text{AcademicExcellence}(x)$

R2: $\text{AcademicExcellence}(x) \text{ AND } \text{Leader}(x) \Rightarrow \text{ScholarshipWinner}(x)$

STARTING GOAL (What we want to prove)

GOAL: $\text{ScholarshipWinner}(\text{Omar})$

BACKWARD CHAINING - Step by Step

FIND RULE Goal: $\text{ScholarshipWinner}(\text{Omar})$

--> Found R2: needs $\text{AcademicExcellence}(\text{Omar}) \text{ AND } \text{Leader}(\text{Omar})$

Sub-goals pushed: $\text{AcademicExcellence}(\text{Omar}) \text{ AND } \text{Leader}(\text{Omar})$



Note: ScholarshipWinner is not a fact. R2 can prove it. Push its conditions.

FIND RULE Goal: $\text{AcademicExcellence}(\text{Omar})$

--> Found R1: needs $\text{Score}(\text{Omar},s) \text{ AND } s \geq 80$

Sub-goals pushed: $\text{Score}(\text{Omar},88) \text{ AND } 88 \geq 80$



Ex 6.4 (Scholarship) — Backward Chaining Diagram (Part 2 of 2)

Note: AcademicExcellence is not a fact. R1 can prove it with a score check.

CHECK FACTS Goal: Score(Omar,88)

--> Score(Omar,88) IS in Working Memory (given exam result)



Note: Score is a known fact for Omar. Proved.

CHECK FACTS Goal: $88 \geq 80$ (arithmetic check)

--> $88 \geq 80$ evaluates to TRUE (numerical verification)



Note: This sub-goal is proved by arithmetic evaluation, not by looking in WM.

PROVED ✓ Goal: AcademicExcellence(Omar)

--> R1 fully satisfied => AcademicExcellence(Omar) PROVED



Note: Score and threshold both verified. R1 fires.

CHECK FACTS Goal: Leader(Omar)

--> Leader(Omar) IS in Working Memory (given recognition)



Note: Leadership is a known fact. Proved immediately.

PROVED ✓ Goal: ScholarshipWinner(Omar)

--> R2 fully satisfied => ScholarshipWinner(Omar) PROVED



Example 6.5: Device Troubleshooting (GOAL FAILS)



PROBLEM STATEMENT

A device management system is asked: "Does Device1 need an OS reinstall?"

Known facts: Device1 is powered on | Device1 has error logs with warning messages

MISSING: NO crash loop has been observed for Device1

Technical knowledge: device ON + crash loop → software fault. software fault + error logs → OS reinstall needed.

Ex 6.5 (Device (Backward)) — FOL Representation

FOL Rules

R1: $\forall d \text{ DeviceOn}(d) \wedge \text{CrashLoop}(d) \rightarrow \text{SoftwareFault}(d)$
R2: $\forall d \text{ SoftwareFault}(d) \wedge \text{LogError}(d) \rightarrow \text{NeedsOSReinstall}(d)$

Facts & Goal

F1: $\text{DeviceOn}(\text{Device1})$
F2: $\text{LogError}(\text{Device1})$
MISSING: $\text{CrashLoop}(\text{Device1})$
not in WM, no rule derives it

WM: { DeviceOn , LogError }

GOAL: PROVE NeedsOSReinstall

Ex 6.5 (Device) — Backward Chaining Trace

Step 1 Goal: NeedsOSReinstall	Is NeedsOSReinstall in WM? NO. Found R2. Conditions: SoftwareFault(Device1) AND LogError(Device1). PUSH SUB-GOALS: [SoftwareFault(Device1), LogError(Device1)]
Step 2 Sub-Goal: SoftwareFault	Is SoftwareFault in WM? NO. Found R1. Conditions: DeviceOn(Device1) AND CrashLoop(Device1). PUSH SUB-GOALS: [DeviceOn(Device1), CrashLoop(Device1)]
Step 3 DeviceOn(Device1)	Is DeviceOn(Device1) in WM? YES ✓ PROVED directly.
Step 4 CrashLoop(Device1)	Is CrashLoop(Device1) in WM? NO ✗ Search KB for any rule with HEAD = CrashLoop... NONE FOUND. CRASHLOOP SUB-GOAL FAILS ✗

Ex 6.5 (Device) — Backward Chaining Trace

Step 5 Propagate Failure

CrashLoop FAILED → R1 cannot fire → SoftwareFault FAILS ✗
SoftwareFault FAILED → R2 cannot fire → NeedsOSReinstall FAILS ✗
Failure propagates all the way up the goal tree.

Ex 6.5 (Device) — Conclusion

✗ GOAL FAILED

NO — we CANNOT conclude that Device1 needs an OS reinstall. GOAL FAILED.

The chain broke at: CrashLoop(Device1) — not in WM, and no rule can derive it.

Failure propagation:

CrashLoop FAILED → SoftwareFault FAILED → NeedsOSReinstall FAILED

Important Lesson: A FAILED GOAL is useful information! It tells us EXACTLY what evidence is missing. A technician should check whether Device1 is actually in a crash loop.

Ex 6.5 (Device) — Backward Chaining Diagram (Part 1 of 2)

BACKWARD CHAINING | Example 5: Does Device1 Need OS Reinstall? (GOAL FAILS)

FOL RULES (Knowledge Base)

R1: $\text{DeviceOn}(d) \text{ AND } \text{CrashLoop}(d) \Rightarrow \text{SoftwareFault}(d)$

R2: $\text{SoftwareFault}(d) \text{ AND } \text{LogError}(d) \Rightarrow \text{NeedsOSReinstall}(d)$

STARTING GOAL (What we want to prove)

GOAL: $\text{NeedsOSReinstall}(\text{Device1})$

BACKWARD CHAINING - Step by Step

FIND RULE Goal: $\text{NeedsOSReinstall}(\text{Device1})$

--> Found R2: needs $\text{SoftwareFault}(\text{Device1}) \text{ AND } \text{LogError}(\text{Device1})$

Sub-goals pushed: $\text{SoftwareFault}(\text{Device1}) \text{ AND } \text{LogError}(\text{Device1})$



Note: NeedsOSReinstall is not a fact. R2 can prove it. Push its conditions.

FIND RULE Goal: $\text{SoftwareFault}(\text{Device1})$

Ex 6.5 (Device) — Backward Chaining Diagram (Part 2 of 2)

Sub-goals pushed: DeviceOn(Device1) AND CrashLoop(Device1)



Note: SoftwareFault is not a fact. R1 can prove it. Push its conditions.

CHECK FACTS Goal: DeviceOn(Device1)

--> DeviceOn(Device1) IS in Working Memory (device is powered on)



Note: Device power state is known. Proved.

FAILED X Goal: CrashLoop(Device1)

--> CrashLoop(Device1) is NOT in WM AND no rule can derive it => FAILED



Note: CrashLoop was never observed. No rule in the KB has CrashLoop as its conclusion. This sub-goal FAILS.

FAILED X Goal: SoftwareFault(Device1)

--> CrashLoop sub-goal failed => R1 cannot fire => SoftwareFault FAILED



Note: Failure of CrashLoop propagates upward. R1 cannot fire without all conditions.

FAILED X Goal: NeedsOSReinstall(Device1)

--> SoftwareFault sub-goal failed => R2 cannot fire => NeedsOSReinstall FAILED



Note: The failure propagates all the way to the top-level goal. Chain breaks.